

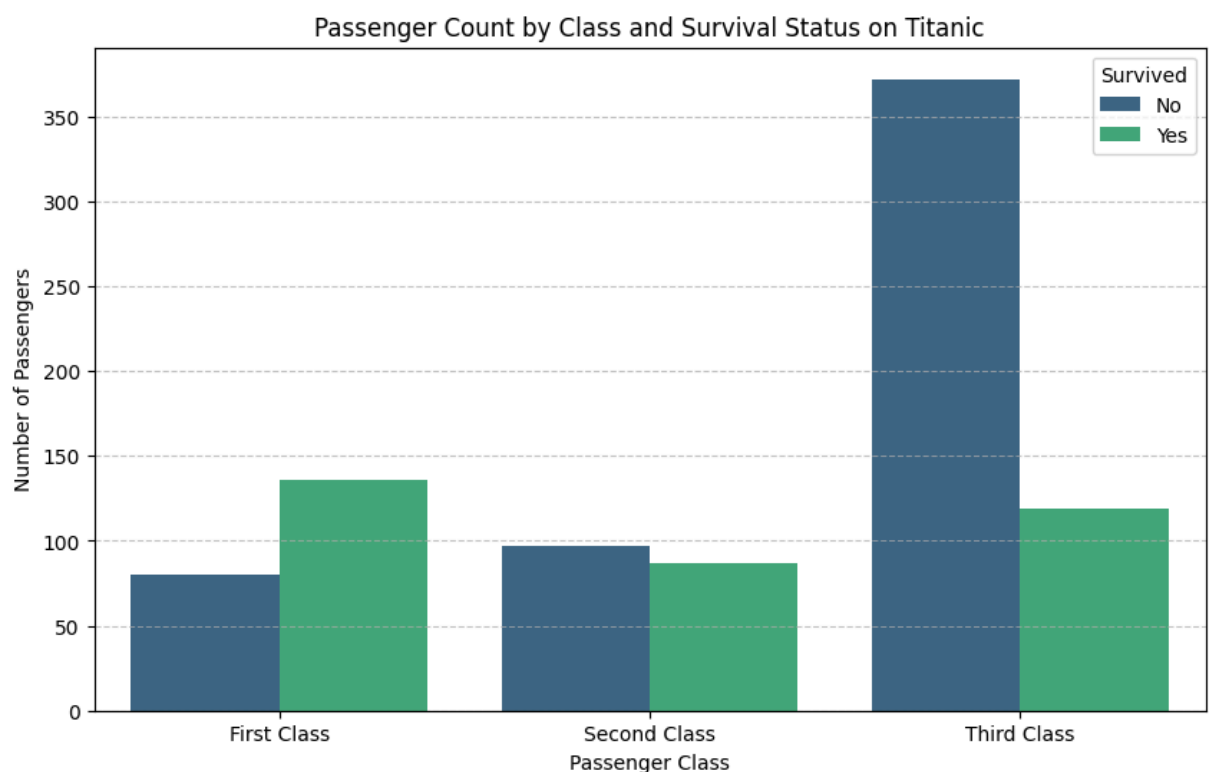
```
import seaborn as sns
import pandas as pd

# Load the titanic dataset
titanic_df = sns.load_dataset('titanic')
```

A bar plot (or bar chart) is a graphical display of data using rectangular bars where the length of each bar is proportional to the value it represents. Class Imbalance: Third Class had the most passengers. Survival Disparity: First Class had the highest survival rate, Second Class was balanced, and Third Class had the lowest survival rate. Socio-economic Impact: Passenger class strongly influenced survival chances.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a bar plot of passenger counts by class, colored by survival status
plt.figure(figsize=(10, 6))
sns.countplot(x='pclass', hue='survived', data=titanic_df, palette='viridis')
plt.title('Passenger Count by Class and Survival Status on Titanic')
plt.xlabel('Passenger Class')
plt.ylabel('Number of Passengers')
plt.xticks([0, 1, 2], ['First Class', 'Second Class', 'Third Class'])
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

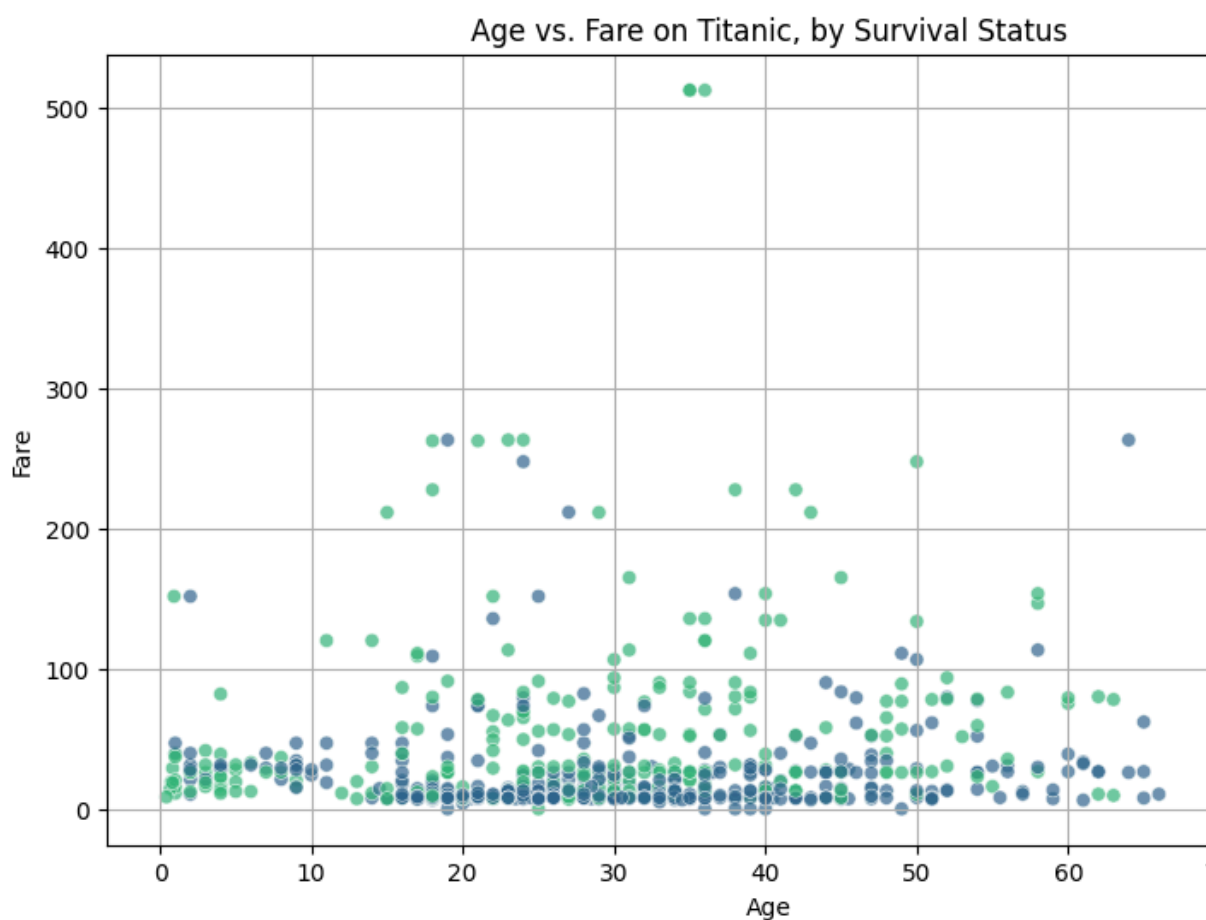


A scatter plot shows individual data points for two numerical variables to reveal relationships or correlations. From the Titanic 'Age vs. Fare' scatter plot Here it shows relation between variables in

## points explaining the spread of data

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a scatter plot of Age vs. Fare, colored by survival status
plt.figure(figsize=(10, 6))
sns.scatterplot(x='age', y='fare', hue='survived', data=titanic_df, palette='viridis', alpha=0.5)
plt.title('Age vs. Fare on Titanic, by Survival Status')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.grid(True)
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```

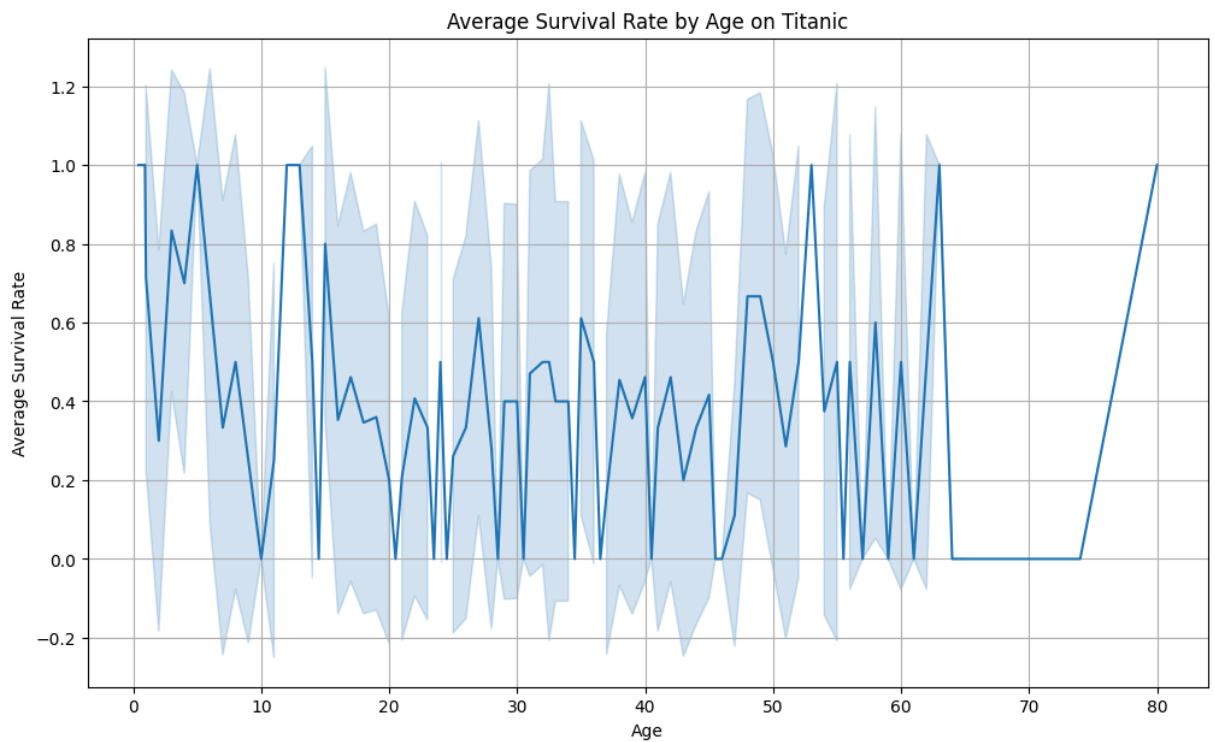


A line graph displays information as a series of data points connected by straight line segments, primarily used to show trends over time or continuous variables.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a line plot of average survival rate by age
plt.figure(figsize=(12, 7))
sns.lineplot(x='age', y='survived', data=titanic_df, estimator='mean', errorbar='sd', palette='viridis')
plt.title('Average Survival Rate by Age on Titanic')
plt.xlabel('Age')
plt.ylabel('Average Survival Rate')
plt.grid(True)
plt.show()
```

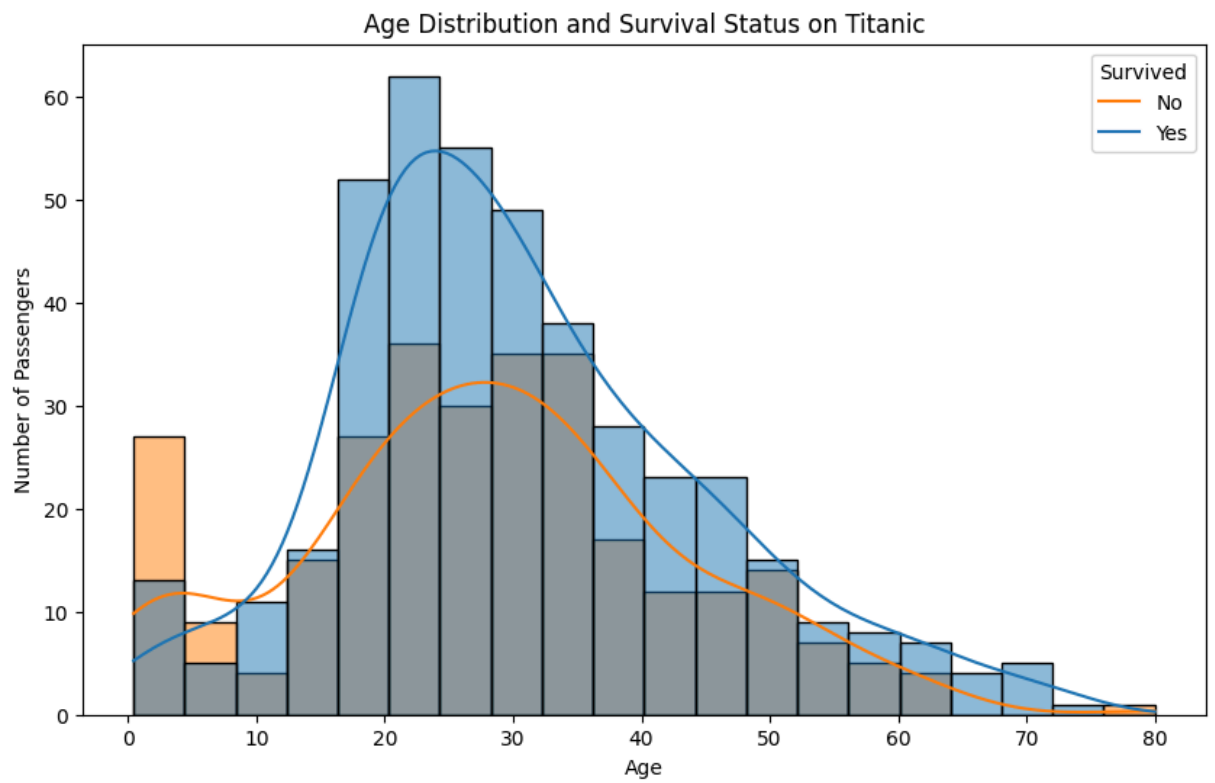
```
/tmp/ipython-input-3399960195.py:6: UserWarning: Ignoring `palette` because no `hue` variable
sns.lineplot(x='age', y='survived', data=titanic_df, estimator='mean', errorbar='sd', palet
```



A histplot (histogram) displays the distribution of a numerical variable by dividing the data into bins and counting the number of observations in each bin. It shows the shape, center, and spread of the data.

```
import matplotlib.pyplot as plt
import seaborn as sns

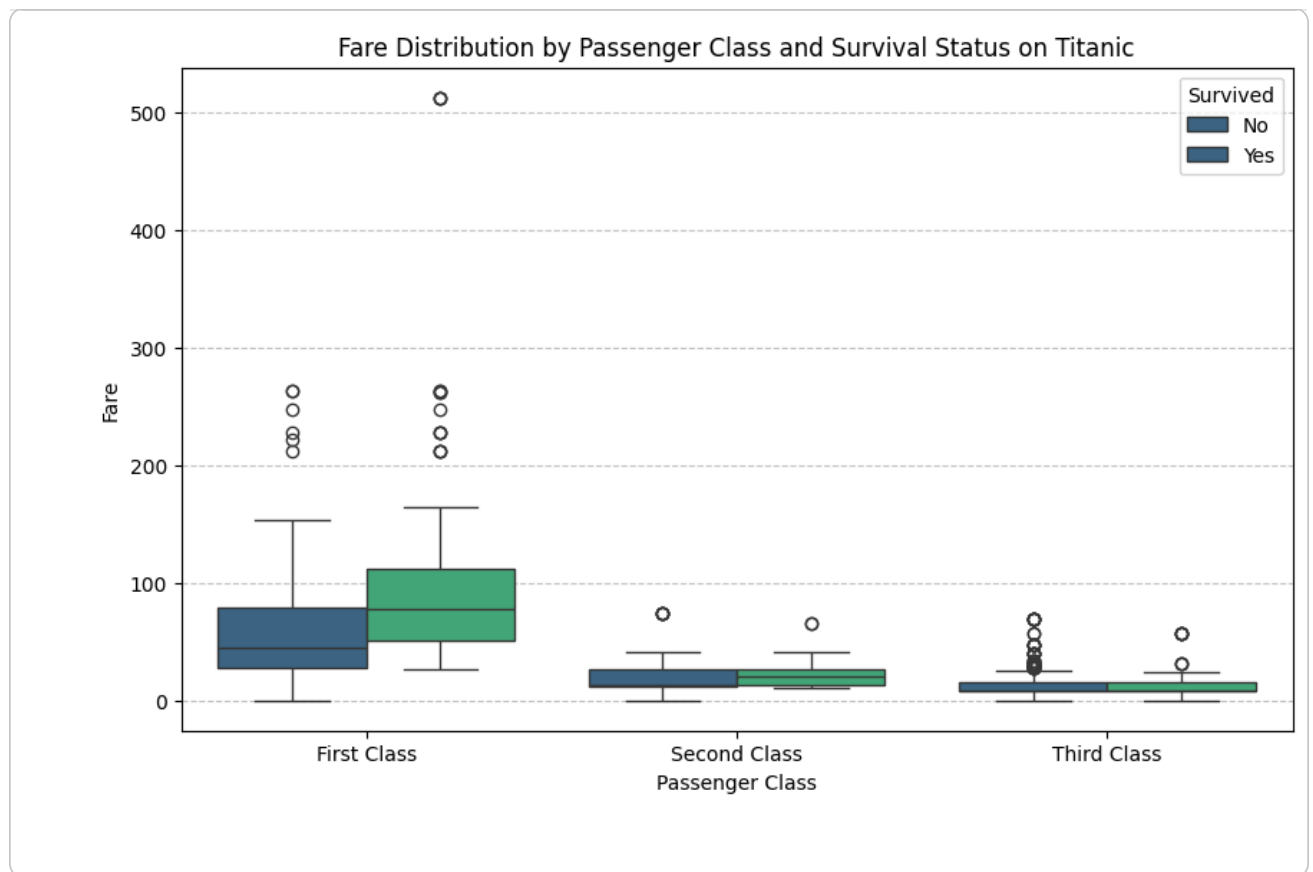
# Create a histplot of Age distribution
plt.figure(figsize=(10, 6))
sns.histplot(data=titanic_df, x='age', kde=True, hue='survived', bins=20)
plt.title('Age Distribution and Survival Status on Titanic')
plt.xlabel('Age')
plt.ylabel('Number of Passengers')
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```



Start coding or [generate](#) with AI.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a boxplot of Fare distribution by Passenger Class, colored by Survival Status
plt.figure(figsize=(10, 6))
sns.boxplot(x='pclass', y='fare', hue='survived', data=titanic_df, palette='viridis')
plt.title('Fare Distribution by Passenger Class and Survival Status on Titanic')
plt.xlabel('Passenger Class')
plt.ylabel('Fare')
plt.xticks([0, 1, 2], ['First Class', 'Second Class', 'Third Class'])
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



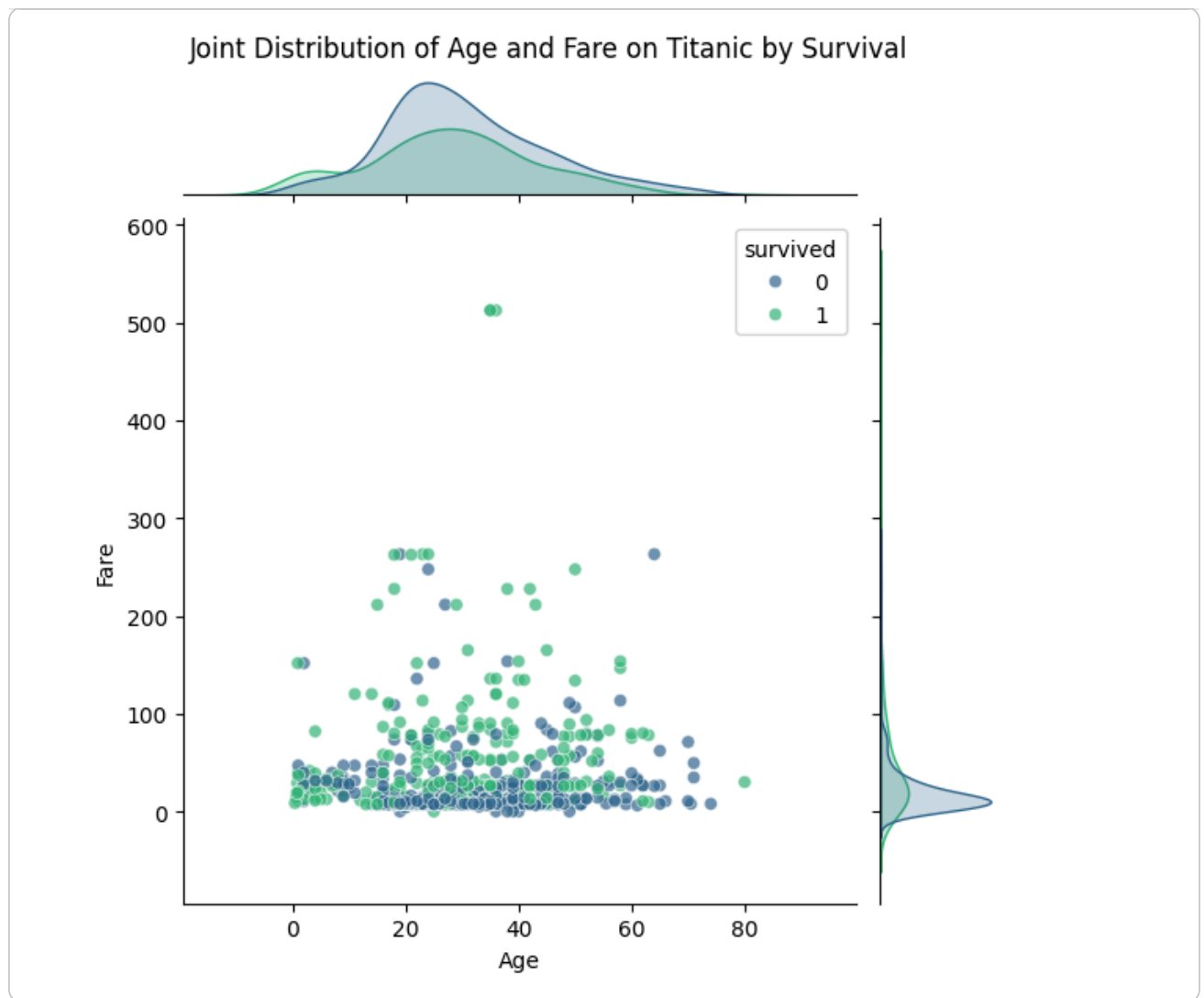
This boxplot visualizes the distribution of 'Fare' for each 'Passenger Class', separated by 'Survival Status'. Each box represents the interquartile range (IQR), with the line inside indicating the median. The 'whiskers' extend to the minimum and maximum values within 1.5 times the IQR, and points beyond are considered outliers. This plot helps us understand the spread and central tendency of fares for different groups, and how they relate to survival.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a joint plot of Age and Fare
g = sns.jointplot(x='age', y='fare', data=titanic_df, kind='scatter', hue='survived', palette='magma')

# Set titles
g.fig.suptitle('Joint Distribution of Age and Fare on Titanic by Survival', y=1.02)
g.set_axis_labels('Age', 'Fare')

plt.show()
```



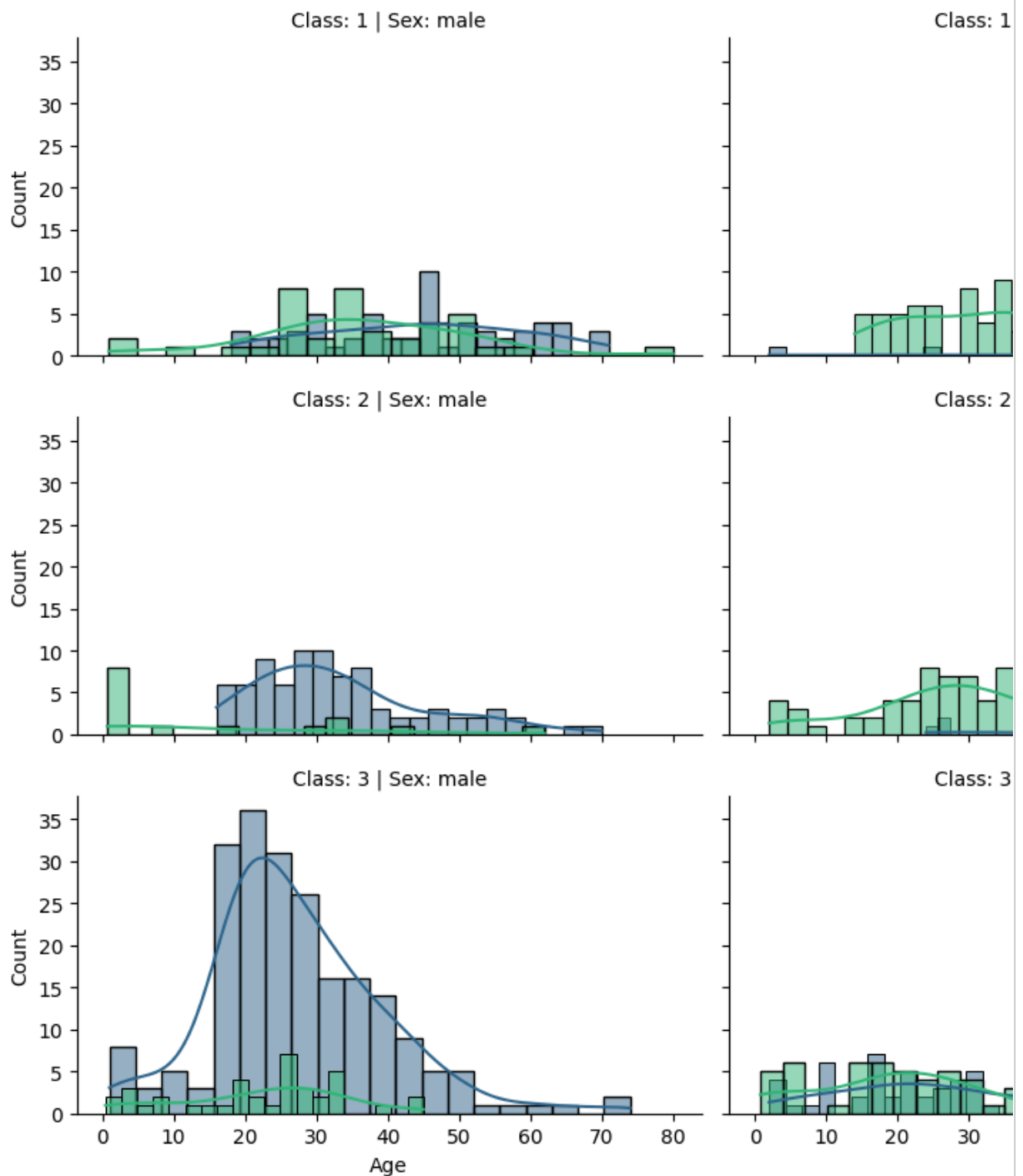
This joint plot shows the relationship between 'Age' and 'Fare' along with their individual distributions. The central scatter plot shows the bivariate relationship, while the histograms on the margins show the univariate distributions of 'Age' and 'Fare' respectively. The `hue='survived'` allows us to distinguish survivors from non-survivors within the scatter plot and marginal histograms.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a FacetGrid to visualize age distribution across sex and passenger class
g = sns.FacetGrid(titanic_df, col='sex', row='pclass', hue='survived', palette='viridis', height=5)
g.map_dataframe(sns.histplot, x='age', kde=True, bins=20)

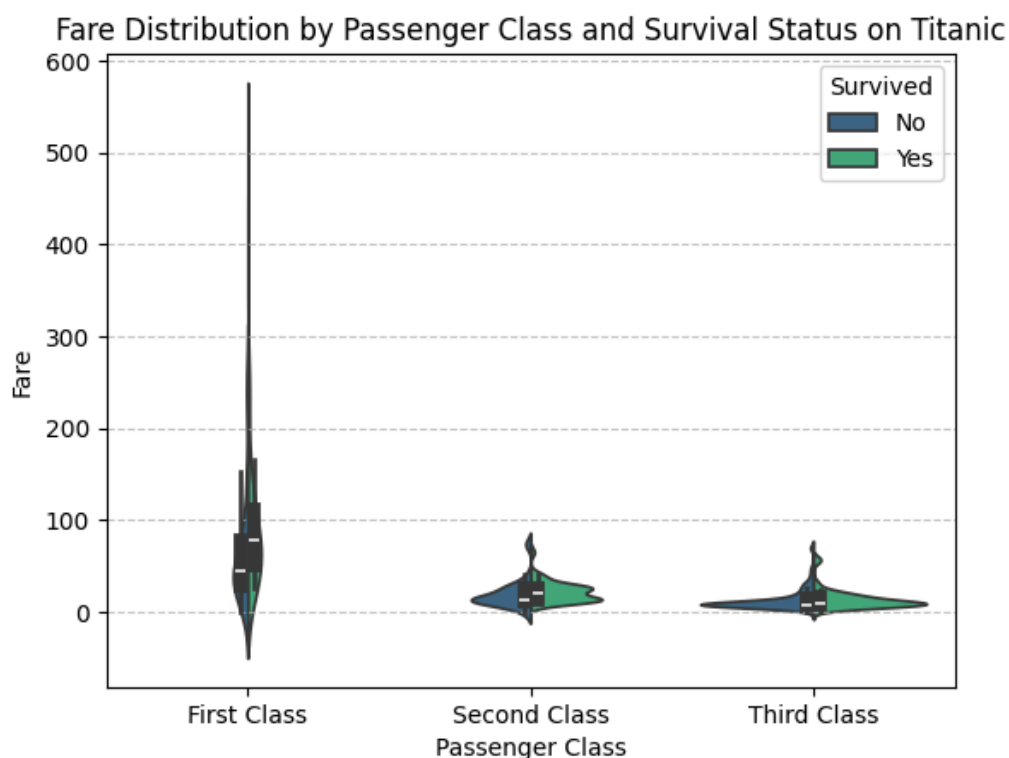
# Add titles and legends
g.set_axis_labels('Age', 'Count')
g.set_titles(col_template='Sex: {col_name}', row_template='Class: {row_name}')
g.add_legend(title='Survived', labels=['No', 'Yes'])
plt.suptitle('Age Distribution by Sex, Passenger Class, and Survival Status', y=1.02, fontsize=12)
plt.tight_layout(rect=[0, 0, 1, 0.98]) # Adjust layout to prevent title overlap
plt.show()
```

## Age Distribution by Sex, Passenger Class, and Survival



```
import seaborn as sns
```

```
ax = sns.violinplot(x='pclass', y='fare', hue='survived', data=titanic_df, palette='viridis')
ax.set_title('Fare Distribution by Passenger Class and Survival Status on Titanic')
ax.set_xlabel('Passenger Class')
ax.set_ylabel('Fare')
ax.set_xticks([0, 1, 2])
ax.set_xticklabels(['First Class', 'Second Class', 'Third Class'])
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles=handles, labels=['No', 'Yes'], title='Survived')
ax.grid(axis='y', linestyle='--', alpha=0.7)
```



This violin plot visualizes the distribution of 'Fare' for each 'Passenger Class', separated by 'Survival Status'. Each 'violin' shape shows the density of the fare values, with the wider sections indicating a higher concentration of data points. The `split=True` argument allows for comparing the distributions of survivors and non-survivors within the same violin for each class, making it easier to see differences in their fare distributions.

This `FacetGrid` displays multiple histograms. Each row represents a passenger class (`pclass`), and each column represents a gender (`sex`). Within each subplot, the age distribution is shown, with different colors indicating survival status. This allows for a very detailed comparison of age, class, gender, and survival.

```
import sklearn as skl
```

Start coding or [generate](#) with AI.

## ✓ Data Preprocessing for Classification

Before we can apply linear classification models, we need to prepare the `titanic_df` dataset. This involves:

1. **Handling Missing Values:** Imputing missing `age` and `embarked` values.
2. **Encoding Categorical Features:** Converting `sex` and `embarked` into numerical representations.
3. **Feature Selection:** Choosing relevant columns for the model.
4. **Splitting Data:** Dividing the dataset into training and testing sets.
5. **Feature Scaling:** Scaling numerical features to ensure all features contribute equally to the model, which is crucial for many linear models.

```
# to create linear model
from sklearn.model_selection import train_test_split
```

```
import pandas as pd

salary_df = pd.read_csv('/Salary_dataset.csv')
```

```
X=salary_df[['YearsExperience']]
y=salary_df['Salary']
X_train, X_test, y_train, y_test= train_test_split(X,y, random_state=26,test_size=0.20)
```

```
X_train.shape
```

```
(24, 1)
```

```
X_test.shape
```

```
(6, 1)
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```
model.score(X_train,y_train)
```

```
0.9460054870434312
```

```
y_pred=model.predict(X_train)
y_valtestpred=model.predict(X_test)
```

```
model.predict(X_train)
```

```
array([[100807.32890847, 125231.5020826 , 53837.76511207, 90474.02487326,
        56655.93893985, 64171.06914728, 111140.63294368, 56655.93893985,
        82019.50338991, 63231.67787135, 72625.59063063, 68868.02552692,
        76383.15573434, 103625.50273626, 40686.28724907, 47262.02618057,
        36928.72214536, 65110.4604232 , 82958.89466584, 123352.71953075,
        38807.50469722, 74504.37318249, 54777.156388 , 93292.19870105])
```

```
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

```
mean_absolute_error(y_test,y_valtestpred)
```

```
3269.3567092524877
```

```
np.sqrt(mean_squared_error(y_test,y_valtestpred))
```

```
np.float64(3925.7392742927045)
```

```
r2_score(y_test,y_valtestpred)
```

```
0.9835849730044816
```

Start coding or [generate](#) with AI.

## Data Preprocessing for Titanic Dataset

We need to re-run the preprocessing steps for the `titanic_df` to prepare it for classification. This includes handling missing values, encoding categorical features, and splitting the data into training and testing sets.

### ✓ Train a Logistic Regression Model

Now, we'll train a simple Logistic Regression model, which is a common linear classifier, and use it to make predictions.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score
import matplotlib.pyplot as plt
```

```
logistic_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(random_state=42, solver='liblinear'))
])
```

```
logistic_model.fit(X_train, y_train)
```

```
y_pred_titanic = logistic_model.predict(X_test)
```

```
print("Logistic Regression model trained and predictions made.")
print(f"Accuracy on test set: {accuracy_score(y_test, y_pred_titanic):.4f}")
```

```
Logistic Regression model trained and predictions made.
Accuracy on test set: 0.7989
```

### ✓ Create Confusion Matrix

Finally, we can generate and display the confusion matrix using the actual test labels and the model's predictions.

```
cm = confusion_matrix(y_test, y_pred_titanic)
```